



UNIVERSIDAD DE EXTREMADURA

CENTRO UNIVERSITARIO DE MÉRIDA

GRADO EN INGENIERÍA INFORMÁTICA EN
TECNOLOGÍAS DE LA INFORMACIÓN

TRABAJO FIN DE GRADO

DISEÑO, DESARROLLO Y DESPLIEGUE DE
APLICACIÓN WEB FULL-STACK PARA EL
ANÁLISIS DE ESTADÍSTICAS DE
VIDEOJUEGOS MEDIANTE EL USO DE API
EXTERNA.

FRANCISCO MONTERO SÁNCHEZ

Mérida, AlgunMes de Esperemos2026



UNIVERSIDAD DE EXTREMADURA

CENTRO UNIVERSITARIO DE MÉRIDA

GRADO EN INGENIERÍA INFORMÁTICA EN
TECNOLOGÍAS DE LA INFORMACIÓN

TRABAJO FIN DE GRADO

DISEÑO, DESARROLLO Y DESPLIEGUE DE
APLICACIÓN WEB FULL-STACK PARA EL
ANÁLISIS DE ESTADÍSTICAS DE
VIDEOJUEGOS MEDIANTE EL USO DE API
EXTERNA.

Autor: Francisco Montero Sánchez
Fdo:

Director: José Manuel Chaves González
Fdo:

Resumen

Aquí va el abstract

Palabras clave -

Abstract

Here we write the abstract

Keywords -

Índice general

Índice de figuras	V
Índice de tablas	VI
1. Introducción	1
1.1. Introducción	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Metodología	5
1.5. Estructura del documento	6
2. Estado del arte	7
2.1. Aplicaciones y herramientas existentes	7
2.2. Plataformas y servicios utilizados	8
2.3. Tecnologías empleadas en este tipo de aplicaciones	10
2.3.1. Tecnologías frontend	10
2.3.2. Tecnologías backend	13
2.3.3. Persistencia y acceso a datos	14
2.3.4. Despliegue	15
2.4. Limitaciones de las soluciones existentes	16
2.5. Aportación de este trabajo	17
3. Marco teórico	19
3.1. Conceptos básicos de League of Legends	19
3.2. Obtención y tratamiento de datos	21
3.2.1. API de Riot	21
3.2.2. Riot ID	22
3.2.3. Plataformas y regiones	23
3.2.4. Partidas y participaciones	24

4. Análisis del problema	25
4.1. Planteamiento del problema	25
4.2. Requisitos del sistema	26
4.3. Arquitectura general y componentes del sistema	29
4.4. Flujo de información	30
5. Diseño del sistema	34
5.1. Diseño de la arquitectura software	34
5.2. Diseño del backend	34
5.3. Diseño del frontend	34
5.4. Diseño de la base de datos	34
5.5. Diseño de la API REST	34
5.6. Diseño del despliegue	34
6. Desarrollo incremental	35
6.1. Resumen general de las iteraciones	35
6.2. Iteración 1	35
6.3. Iteración 2	35
7. Evaluación de la aplicación final	36
7.1. Entorno de despliegue	36
7.2. Casos de uso y validación funcional	36
7.3. Comprobación del cumplimiento de requisitos	36
7.4. Análisis crítico del desarrollo realizado	36
7.5. Limitaciones del sistema	36
8. Conclusiones y trabajo futuro	37
8.1. Conclusiones	37
8.2. Grado de cumplimiento de los objetivos	37
8.3. Aportaciones del trabajo	37
8.4. Limitaciones actuales	37
8.5. Líneas de trabajo futuro	37
Referencias	38

Índice de figuras

2.1. Encuesta sobre frameworks y tecnologías web	12
2.2. Encuesta bases de datos	15
3.1. API de Riot. DTO	22
3.2. Endpoint MATCH-V5	23
4.1. Diagrama de componentes de alto nivel	29
4.2. Diagrama de flujo. Perfil de usuario	31
4.3. Diagrama de flujo. Builds de profesionales	32

Índice de tablas

4.1. Requisitos funcionales.	27
4.2. Requisitos no funcionales.	28
4.3. Requisitos funcionales de ampliación.	28

Capítulo 1

Introducción

1.1. Introducción

La industria de los videojuegos ha crecido enormemente en los últimos años. Desde la aparición de los primeros títulos en los años 50, estos han ido evolucionando constantemente, mejorando aspectos gráficos, de jugabilidad e incluso descubriendo nuevos géneros.

A comienzos del siglo, el aumento del público interesado en los videojuegos llevó a que se crearan ligas y torneos similares a los existentes en los deportes tradicionales. Así surgieron los E-Sports (Deportes Electrónicos), consolidándose como una disciplina competitiva que exige la actualización constante de los jugadores.

Este proyecto se centra en *League of Legends* (LoL), uno de los videojuegos más jugados en la actualidad. Este es un videojuego competitivo perteneciente al género *Multiplayer Online Battle Arena* (MOBA), donde dos equipos formados por cinco jugadores, distribuidos en diferentes roles, compiten por destruir la base del rival.

Los MOBA son videojuegos con una multitud de combinaciones posibles. Entre ellos, el LoL consta de un amplio conjunto de campeones (personajes jugables), habilidades y objetos, lo que genera un amplio espacio de decisiones para el jugador.

El número de todas las combinaciones posibles, junto con la actualización constante de objetos y personajes, hace que la complejidad del videojuego sea muy elevada para los jugadores casuales que no pueden dedicar el tiempo necesario para mantenerse actualizados de forma constante.

Para afrontar este problema, la empresa dueña del videojuego, Riot Games, proporciona una API que permite obtener datos relacionados con cuentas, partidas y otros elementos del juego. Esto permite la creación de apli-

caciones que analizan las mejores combinaciones posibles en cada momento, facilitando la experiencia del usuario casual.

Este trabajo se basa en la creación de una aplicación web a la que los usuarios puedan acudir para consultar perfiles, visualizar su historial de partidas, comprobar su estado de partida actual y analizar builds (conjunto de objetos comprados) de jugadores profesionales. Además, el proyecto contempla la incorporación de estadísticas avanzadas para los distintos campeones y jugadores.

El desarrollo de esta aplicación permitirá a los usuarios consultar las mejores opciones posibles en tan solo unos segundos. De esta forma, podrán consultar información relevante de forma rápida, reduciendo el esfuerzo necesario para mantenerse actualizados y facilitando la toma de decisiones dentro del videojuego.

Dado el alcance del problema y las casi infinitas funcionalidades que se pueden implementar, el desarrollo del proyecto se ha planteado mediante una metodología iterativa e incremental. De esta forma, en una primera iteración se desarrolla el Producto Mínimo Viable (MVP) y en las siguientes se implementan funcionalidades de interés para el usuario final.

Nota: Desde pequeño nos enseñan que cada apartado debería acabar con un resumen, por ejemplo: En conclusión... El tema es que esto es muy de chatgpt, no sé si debería ponerlo o simplemente dejarlo así. En general, no se si termino bien todos los apartados o debo poner un párrafo de resumen final.

1.2. Motivación

Tras conocer y analizar el entorno de los videojuegos competitivos, específicamente los MOBA, durante más de una década, se ha observado la necesidad de que la comunidad cuente con un sitio fijo al que acudir en busca del conocimiento necesario.

A diferencia de otros videojuegos, como los *First Person Shooter* (FPS), el caso de Counter-Strike o los *Battle Royale* como Fortnite, los MOBA dan mucha importancia a la estrategia, la coordinación del equipo y la toma de decisiones durante la partida. Por este motivo, suelen presentar una barrera de entrada más elevada y requieren un mayor tiempo de aprendizaje.

La motivación principal de este proyecto es facilitar el aprendizaje continuo de los jugadores experimentados del LoL y reducir la barrera de entrada para los jugadores nuevos que, sin ayuda, podrían tardar años en aprender los fundamentos del juego y estar al día.

La complejidad al aprender los diversos aspectos del videojuego hace que existan otros factores que motivan la existencia de este proyecto:

- **Conocimiento del META:** Con la actualización constante de los videojuegos competitivos, surge el *Most Efficient Tactics Available* (META). En un contexto donde cada dos semanas cambian los personajes, objetos y características del entorno, surge la necesidad de conocer las mejores opciones en todo momento. La forma más rápida de satisfacer esto es tener un sitio al que acudir cuando se desee conocer el META actual.
- **Revisión de estadísticas:** La mejora en un entorno competitivo requiere un análisis detallado del jugador. Muchas veces, es necesario consultar estadísticas avanzadas, como tendencias, puntos fuertes, débiles o el desempeño en cada partida.
- **Centralización de la información:** La información actual recopilada por terceros se encuentra muy distribuida en diferentes aplicaciones. Unas se especializan en consultar perfiles, otras analizan el rendimiento del jugador mediante inteligencia artificial y otras ofrecen recomendaciones a partir de los datos existentes, pero no existe un sitio al que los jugadores, especialmente los más novatos, puedan acceder para consultar la información básica de una forma sencilla.

La existencia de un sitio al que acudir en busca del conocimiento de forma sencilla satisface todos estos factores y es de gran ayuda para la comunidad.

Junto con las motivaciones generales de la existencia de este proyecto, también surge una motivación personal en la forma en que se llevará a cabo.

A lo largo de la carrera se han estudiado distintas tecnologías y fundamentos del desarrollo software. Este trabajo permite complementar esa base con herramientas actuales utilizadas habitualmente en el desarrollo de aplicaciones web reales. Actualmente, la mayor parte de las aplicaciones y servicios web se basan en el uso de APIs para el intercambio de información. Además, el proyecto ofrece la oportunidad de desarrollar una aplicación full-stack que integra frontend, backend y base de datos.

Fuente frontend y backend: <https://aws.amazon.com/es/compare/the-difference-between-aws-elasticbeanstalk-and-aws-lambda>

1.3. Objetivos

El objetivo principal es diseñar, desarrollar y desplegar una aplicación web full-stack orientada a la consulta y análisis de información del videojuego

League of Legends, a partir de datos obtenidos de la API pública de Riot Games, siguiendo una metodología de desarrollo iterativa e incremental.

Para lograr el objetivo principal, se han definido los siguientes objetivos específicos:

- **Diseñar un modelo de datos** para guardar la información de los jugadores, partidas, campeones, builds y jugadores profesionales. Este modelo de datos sirve para que la información pueda reutilizarse sin repetir las mismas consultas a la API externa.
- **Desarrollar un backend** que se conecte con la API de Riot, procese la información recibida, la almacene en la base de datos y la exponga mediante una API REST.
- **Desarrollar un frontend web** que permita al usuario final consultar perfiles, historial de partidas y builds de jugadores profesionales de una forma clara y sencilla.
- **Implementar un primer incremento funcional** que actúe como MVP. De esta forma, se dispone de una base sobre la que agregar funcionalidades en los siguientes incrementos.
- **Configurar un entorno de ejecución local** que permita levantar los distintos componentes de la aplicación, incluyendo frontend, backend y base de datos.

Además, para complementar la base del proyecto, se ha establecido una serie de objetivos de ampliación:

- **Añadir estadísticas avanzadas para campeones y jugadores** en incrementos posteriores al primero.
- **Automatizar la actualización de la información** mediante tareas automáticas en el backend.
- **Preparar el paso a producción** publicando el frontend y el backend en servicios de hosting y configurando un dominio propio para su acceso público.

1.4. Metodología

El desarrollo del proyecto se ha llevado a cabo siguiendo una metodología de desarrollo iterativa e incremental. Esta metodología permite construir una base funcional desde el primer incremento y ampliar posteriormente las funciones de la aplicación sin rehacer completamente la arquitectura base del sistema.

La elección de esta metodología se debe a que el desarrollo del proyecto conlleva el aprendizaje de nuevas herramientas y tecnologías. En caso de haber usado una metodología tradicional como el desarrollo en cascada, se podrían haber planificado o diseñado elementos que posteriormente resultasen difíciles de cumplir. En cambio, al dar un enfoque iterativo e incremental se consigue solucionar este problema avanzando de forma progresiva con incrementos e iteraciones.

Nota: No sé si poner aquí algo como: no se han elegido metodologías novedosas ágiles como Scrum porque están diseñadas para trabajar en equipo y yo estoy solo.

El desarrollo evoluciona mediante distintos incrementos funcionales. Cada incremento añade nuevas funcionalidades a la versión previa del sistema; de esta manera, el proyecto crece de forma progresiva en lugar de desarrollarse completamente desde el principio. Esto permite establecer una base funcional que se puede mejorar y completar con el transcurso de los incrementos.

A su vez, cada incremento se desarrolla de forma iterativa. En cada uno de ellos se siguen todas las etapas del proceso de creación de software: análisis, diseño, implementación y validación. Gracias a este proceso no solo se añaden nuevas funcionalidades sino que también se puede refinar la base anterior en caso de ser necesario.

Fuente metodología: <https://blog.jmbeas.es/2019/09/22/desarrollo-iterativo-e-inc>

El primer incremento del proyecto se centra en el desarrollo del MVP. El MVP consiste en una versión inicial del software que incluye solo las características esenciales para satisfacer al usuario y evaluar la viabilidad del proyecto. De esta forma, el primer incremento construye una primera versión funcional que sirve como base para su posterior evolución.

Fuente MVP: <https://www.impulsa-empresa.es/diccionario/producto-minimo-viable/>

En este primer incremento se establece la arquitectura general de la aplicación: se crea el modelo de datos, la integración con la API de Riot y la estructura básica del backend y del frontend. A partir de esta base se podrán incluir nuevas funcionalidades de una forma más rápida y sencilla.

Los siguientes incrementos se centran en la creación de nuevas funcionalidades. Algunas de las funcionalidades previstas son la incorporación de estadísticas avanzadas y la automatización de la actualización de la información mediante tareas programadas en el backend.

1.5. Estructura del documento

Este documento se estructura en los siguientes capítulos:

- **Capítulo 2. Estado del Arte:** Revisa las aplicaciones existentes basadas en APIs de videojuegos y las tecnologías más utilizadas en este tipo de sistemas. Además, se expone la aportación de este trabajo respecto a las soluciones ya existentes.
- **Capítulo 3. Marco teórico:** Introduce los conceptos generales y el vocabulario técnico relacionados con el proyecto con el objetivo de aportar una base de conocimiento para entender el trabajo realizado.
- **Capítulo 4. Análisis del problema y metodología empleada:** Describe el problema que se pretende resolver, los requisitos de la aplicación, los componentes del sistema y la arquitectura general.
- **Capítulo 5. Diseño del sistema:** Detalla las tecnologías utilizadas y se justifican las decisiones tomadas durante el diseño. También se describen la base de datos, clases y otros componentes de la solución.
- **Capítulo 6. Desarrollo incremental:** Explica la evolución del proyecto a través de los sucesivos incrementos, describiendo las funcionalidades implementadas en cada uno de ellos y la mejora respecto a la versión anterior.
- **Capítulo 7. Evaluación de la aplicación final:** Presenta el despliegue de la versión final, sus características técnicas y distintos casos de uso para validar el funcionamiento de la aplicación.
- **Capítulo 8. Conclusiones y trabajo futuro:** Recoge las principales conclusiones del trabajo, las aportaciones realizadas y los conocimientos adquiridos durante su desarrollo. Finalmente, se plantean posibles mejoras para la evolución del proyecto.

Capítulo 2

Estado del arte

Nota: No sé si introducir hablando de 'En este capítulo...' o pasar a la primera sección directamente como hice con la introducción, ya me dirás como lo ves mejor

En este capítulo se revisan las aplicaciones y herramientas existentes relacionadas con el análisis de datos de videojuegos, especialmente en el caso del LoL. También se presentan las plataformas y servicios más utilizados, así como las tecnologías más habituales en el desarrollo de este tipo de sistemas. El objetivo final es definir este trabajo en el contexto actual y justificar su aporte frente a las soluciones ya existentes.

2.1. Aplicaciones y herramientas existentes

En el contexto actual del LoL, existen numerosas aplicaciones orientadas a satisfacer las distintas necesidades de los jugadores. Algunas de ellas están centradas en consultas de perfiles e historiales, permitiendo buscar jugadores y revisar su historial reciente. Entre este tipo de aplicaciones, la más destacada es OP.GG, que se ha consolidado como la mayor referencia para el seguimiento de cuentas y la consulta rápida de información. Esta plataforma comenzó a funcionar en el año 2012 como buscador de estadísticas del LoL y actualmente cuenta con 16 videojuegos diferentes, sumando 14 millones de visitas diarias y más de 55 millones de visitantes únicos mensuales.

Fuente usuarios opgg: <https://op.gg/es/lol/about>

Junto a estas herramientas, también existen otras aplicaciones especializadas en campeones, builds y META. Su función principal es mostrar las mejores combinaciones de objetos, runas, orden de habilidades y estadísticas para cada campeón. Este tipo de aplicaciones es realmente útil para estar

actualizado en todo momento y elegir la mejor combinación. Una de las más destacadas es Blitz.gg, cuyo perfil de LinkedIn indica que la aplicación cuenta con más de 8 millones de usuarios activos. Además, este tipo de aplicaciones suele ofrecer un overlay que ayuda a tomar decisiones en tiempo real.

Fuente usuarios blitzgg: <https://www.linkedin.com/company/theblitzapp/about/>

Nota: No sé si poner como fuente el linkedin de la empresa es fiable, no he encontrado una fuente "fiable" de información.

Por último, también hay herramientas orientadas al análisis competitivo y al seguimiento de jugadores profesionales. Estas permiten consultar partidas recientes, campeones utilizados y builds de jugadores profesionales. Gracias a ello, los usuarios pueden acceder a una fuente de información actualizada y fiable, lo que facilita conocer las estrategias utilizadas en el entorno competitivo del videojuego. Entre ellas destaca Probuild Stats, que recopila información sobre builds empleadas por jugadores profesionales.

Nota: En el caso de op.gg y blitz.gg puse el link porque es corto y muchas veces el gg en este tipo de aplicaciones va junto al nombre, pero en el caso de probuildstats.com que hago? puse el nombre sin el .com pero no se muy bien como poner eso, si lo pongo en negritas, lo dejó así o en cursiva.

Continuación de nota: Ahora mismo las palabras en inglés van en cursiva y estoy poniendo la palabra/expresión completa y luego las siglas entre paréntesis como hacíamos en asle, supongo que así está bien también aquí? Ejemplo: League of Legends en cursiva (LoL).

2.2. Plataformas y servicios utilizados

El funcionamiento de este tipo de aplicaciones depende en gran medida de la disponibilidad de fuentes de datos fiables y actualizadas. En el caso de LoL, la fuente principal de información es la API de Riot, un servicio oficial que permite acceder a información relacionada con cuentas de usuarios, partidas y diferentes elementos del juego. Gracias a esta API, los desarrolladores pueden construir aplicaciones y herramientas externas capaces de procesar la información del juego para satisfacer las necesidades de la comunidad del videojuego.

La API de Riot utiliza un sistema de *API Keys* para autenticar las solicitudes realizadas por aplicaciones externas. Riot ofrece diferentes tipos de claves según el uso y la fase de desarrollo del producto:

- ***Development API Key***: es la clave más básica y se puede obtener gratuitamente desde la web oficial de Riot Developer utilizando una cuenta de Riot. Esta clave está pensada para pruebas y desarrollo inicial, por lo que tiene una limitación de 100 llamadas a la API cada 2 minutos y, además, debe renovarse diariamente.
- ***Personal API Key***: está orientada a proyectos personales, pequeñas comunidades privadas o trabajos de investigación. Para obtenerla, es necesario registrar el producto en Riot Developer y proporcionar una descripción detallada del proyecto. A diferencia de la clave de desarrollo, no debe renovarse diariamente, pero sigue teniendo los mismos límites y no puede utilizarse para aplicaciones públicas.
- ***Production API Key***: está destinada a aplicaciones públicas en producción. Para obtener acceso a esta clave, es necesario enviar una solicitud a Riot Games explicando el funcionamiento de la aplicación, una descripción detallada del proyecto y la URL de la aplicación en producción. Este tipo de clave tiene límites mucho más amplios, pensados para soportar el tráfico de usuarios reales. En caso de llevar el proyecto a producción, sería necesario solicitar esta clave.

Nota: Cuando hablo de estos términos en inglés que tienen traducción al español, los trato solo en inglés en cursiva, los pongo primero en inglés y luego en español entre paréntesis, y ya lo trato en español o que hago?

Además de la API de Riot, también es necesario utilizar *Data Dragon* como fuente de recursos estáticos. Este servicio proporciona información y recursos visuales del juego, como campeones, objetos o iconos de perfil. A diferencia de la API de Riot, que permite obtener datos, *Data Dragon* se utiliza para acceder a la información estática del videojuego.

Estos recursos están organizados por versión del juego e idioma, lo que permite trabajar con datos actualizados a cada parche en el lenguaje deseado. Por este motivo, *Data Dragon* resulta muy útil en aplicaciones de análisis o consulta de datos del LoL, ya que permite complementar la información obtenida mediante la API con nombres, descripciones e imágenes que mejoran la presentación de los datos.

Fuente riot developer (data dragon viene aqui): <https://developer.riotgames.com/>

2.3. Tecnologías empleadas en este tipo de aplicaciones

El desarrollo de aplicaciones web ha evolucionado mucho desde las primeras páginas estáticas hasta los sistemas actuales, donde es habitual separar la interfaz de usuario, la lógica del servidor y el acceso a datos. En este contexto, las APIs REST se han convertido en una de las formas más utilizadas para comunicar aplicaciones a través de internet, ya que están basadas en interacciones cliente-servidor mediante peticiones HTTP.

Fuente API REST: <https://www.redhat.com/es/topics/api/what-is-a-rest-api>

Este modelo encaja perfectamente en aplicaciones basadas en videojuegos, ya que muchas de ellas dependen de APIs externas para acceder a datos del propio juego. Un ejemplo claro es la API de Riot Games, mencionada anteriormente, que permite obtener información actualizada sobre cuentas, partidas y otros elementos de LoL.

En este tipo de aplicaciones, suele emplearse una arquitectura full-stack, separando la interfaz de usuario, la lógica del servidor y el almacenamiento de datos. Esta separación facilita el mantenimiento del sistema y permite que cada parte se desarrolle de forma independiente.

Nota: Todo esto de full stack ya lo expliqué tal cual al final de la motivación, no sé si repetir algo está bien o digo algo como como se mencionó anteriormente igual que hice con la descripción de la API de Riot en el parrafo anterior.

2.3.1. Tecnologías frontend

En la parte frontend, la tendencia más común es el desarrollo de aplicaciones SPA, es decir, aplicaciones de una sola página. En este modelo, el navegador carga un único documento principal y el contenido se actualiza dinámicamente mediante JavaScript, sin necesidad de recargar la página en cada interacción.

Para desarrollar este tipo de interfaces se utilizan tecnologías como React, Vue o Angular. Estas herramientas permiten organizar la interfaz mediante componentes reutilizables, lo que facilita la división de la aplicación en partes más pequeñas. Este enfoque resulta útil en aplicaciones que obtienen datos

de APIs externas, ya que permite actualizar de forma controlada las partes de la interfaz que dependen de la información consultada.

Nota: No sé hasta qué punto debo poner fuentes. Pongo a continuación de donde saco que son las más utilizadas, aunque es sabido por todo el mundo sin necesidad de tener una fuente, la cosa es, debería poner también la web de cada tecnología como fuente de que sirven para interfaces basadas en componentes? lo veo como poner por poner.

A partir de la encuesta general de tecnologías web realizada por Stack Overflow en 2025, se puede ver el uso de algunas de las principales tecnologías utilizadas en frontend. Como muestra la Figura 2.1, React aparece con un 44,7% de uso entre los más de 23000 encuestados, mientras que Angular alcanza un 18,2% y Vue.js un 17,6%. Aunque la encuesta no se limita únicamente al desarrollo frontend e incluye otras tecnologías de backend, estos datos establecen a React, Angular y Vue.js como las opciones más relevantes dentro del desarrollo de interfaces web modernas.

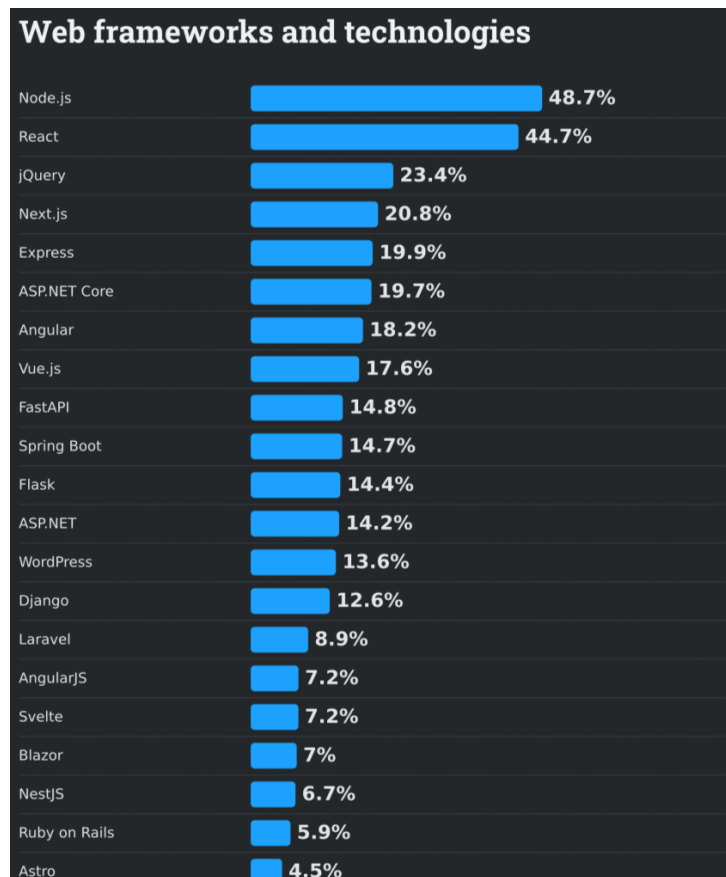


Figura 2.1: Encuesta sobre frameworks y tecnologías web

Fuente SPA: <https://www.unir.net/revista/ingenieria/single-page-application-que>

Fuente encuesta: <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-webframe-webframe>

Junto a JavaScript, también es frecuente el uso de TypeScript. Esta tecnología añade sintaxis de tipos sobre JavaScript, lo que ayuda a detectar errores durante el desarrollo y mejora la mantenibilidad del código. En aplicaciones que usan APIs externas, el uso de tipos permite representar de forma más clara la estructura de los datos recibidos.

Nota: Aquí lo mismo, pongo fuente de que es javascript y que es typescript? es que si me lio a poner referencias acabo con 10 páginas de referencias.

Además del framework o librería principal, las aplicaciones frontend suelen utilizar herramientas de construcción como Vite. Estas herramientas permiten ejecutar el proyecto durante el desarrollo de forma rápida y generar

una versión optimizada para producción. Para ello, Vite incluye un servidor de desarrollo con recarga rápida de módulos y un sistema de compilación que empaqueta el código. Además, puede integrarse con distintos frameworks y ampliarse mediante plugins.

Fuente Vite: <https://vite.dev/guide/>

En cuanto al diseño visual, existen distintas formas para construir una interfaz de calidad. Frameworks como Bootstrap ofrecen componentes y estilos preparados para crear interfaces responsive, mientras que herramientas como Tailwind CSS proporcionan clases que permiten construir diseños personalizados directamente desde el código HTML. Estas herramientas son muy útiles para tener una interfaz de calidad sin necesidad de escribir grandes hojas de estilo CSS.

2.3.2. Tecnologías backend

En la parte backend, lo más común es construir una API propia que actúe como intermediaria entre el frontend, la base de datos y las APIs externas. De esta forma, el frontend no se comunica directamente con los servicios externos, sino que lo hace a través del backend de la aplicación. Esta división permite centralizar la lógica de negocio en un mismo lugar para controlar mejor los errores, gestionar la persistencia de datos y proteger información sensible.

Para desarrollar backends, existen varias tecnologías ampliamente utilizadas. Dentro del ecosistema de Java, destaca Spring Boot, un framework para crear aplicaciones y servicios web. También son frecuentes aplicaciones basadas en Node.js, como Express; frameworks de Python, como FastAPI; y tecnologías del entorno .NET, como ASP.NET Core.

Como se muestra en la Figura 2.1, la encuesta de Stack Overflow de 2025 también refleja el uso de varias de estas tecnologías: Node.js aparece con un 48,7 %, Express con un 19,9 %, ASP.NET Core con un 19,7 %, FastAPI con un 14,8 %, y Spring Boot, con un 14,7 %. Estos datos muestran las diferentes alternativas para el desarrollo de aplicaciones web y APIs en ecosistemas como JavaScript, Java, Python o .NET.

Además de elegir una tecnología concreta, en el desarrollo backend también es importante establecer una organización adecuada. En aplicaciones pequeñas puede encontrarse toda la lógica en una única capa, mezclando la gestión de peticiones, el procesamiento de datos y el acceso a la base de datos. Aunque esto puede ser válido para prototipos, dificulta el mantenimiento cuando la aplicación crece.

Por este motivo, en los backends modernos es habitual separar responsabilidades mediante una arquitectura por capas. Normalmente se distinguen controladores, encargados de gestionar las peticiones HTTP; servicios, donde se concentra la lógica de negocio; repositorios, encargados del acceso a datos; y modelos o entidades, que representan la información del sistema. En aplicaciones basadas en APIs externas, también pueden añadirse componentes específicos para comunicarse con dichos servicios y DTOs para controlar la estructura de los datos que se reciben o envían.

2.3.3. Persistencia y acceso a datos

Las bases de datos permiten almacenar información para ser reutilizada posteriormente. En el desarrollo de aplicaciones web es habitual encontrar bases de datos relacionales como PostgreSQL, MySQL o MariaDB, que organizan la información en tablas y permiten definir relaciones entre ellas. Este modelo resulta muy útil cuando los datos tienen una estructura clara y están relacionados entre sí, como puede ocurrir con usuarios, estadísticas, partidas o elementos asociados a una cuenta.

Además de las bases de datos relacionales, existen bases de datos NoSQL como MongoDB, que pueden ser útiles cuando la información tiene una estructura flexible que cambia frecuentemente. También son útiles las bases de datos en memoria, como Redis, para almacenar temporalmente información consultada con frecuencia. Esto permite reducir consultas repetidas a la base de datos, algo clave en las aplicaciones que dependen de APIs con límites de uso.

Como se puede ver en la Figura 2.2, la encuesta de Stack Overflow de 2025 muestra que PostgreSQL es una de las bases de datos más utilizadas, con un 55,6 % de uso entre los encuestados. También aparecen MySQL, con un 40,5 %, SQLite, con un 37,5 %, Microsoft SQL Server, con un 30,1 %, Redis con un 28 % y MongoDB, con un 24 %. Estos datos muestran que las bases de datos relacionales son las más utilizadas, aunque también se utilizan bases de datos NoSQL o en memoria.

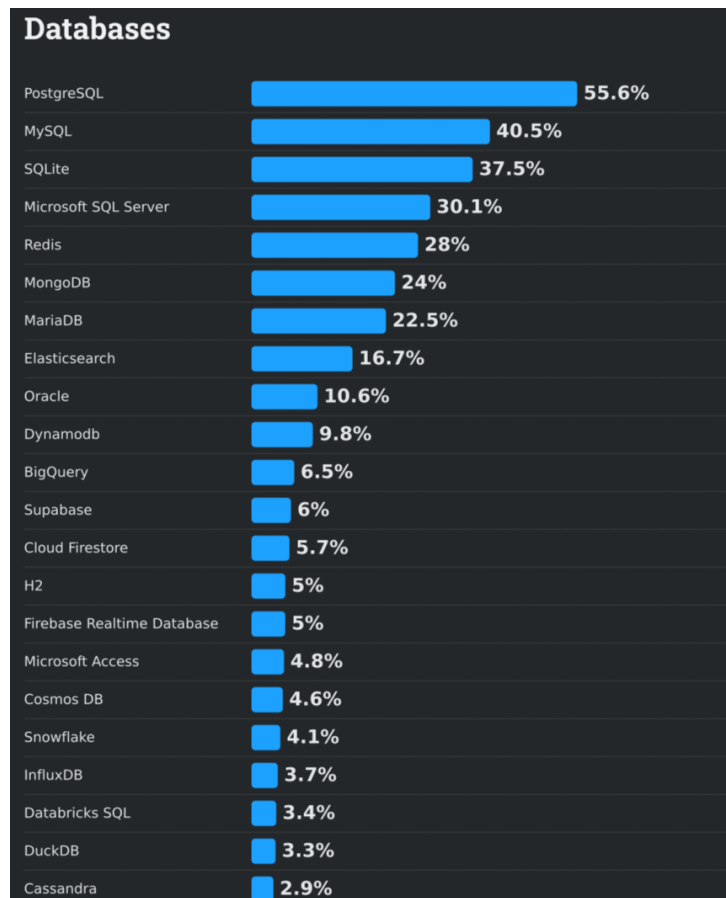


Figura 2.2: Encuesta bases de datos

2.3.4. Despliegue

Durante el desarrollo local de una aplicación full-stack, es habitual ejecutar por separado las distintas partes del sistema. En el frontend, herramientas como Vite permiten levantar un servidor rápido que permite ver cambios en la interfaz durante el desarrollo. En el backend y la base de datos, es habitual utilizar contenedores para ejecutar servicios de forma aislada y reproducible, evitando problemas al cambiar entre equipos.

En aplicaciones formadas por varios servicios, herramientas como Docker y Docker Compose facilitan la ejecución de las distintas partes de la aplicación durante el desarrollo. Esto simplifica la puesta en marcha del sistema completo en local y permite gestionar de forma separada servicios como el backend y la base de datos.

Para publicar una aplicación en un entorno accesible desde internet, existen distintas alternativas. En el caso del frontend, plataformas como Vercel

o Netlify están orientadas a simplificar el despliegue de aplicaciones web, conectándose directamente al repositorio de GitHub. Para el backend, existen plataformas como Render o Railway que permiten desplegar servicios web o APIs sin tener que gestionar manualmente la infraestructura.

Además de las opciones sencillas, existen plataformas cloud más completas y profesionales, como Amazon Web Services, Microsoft Azure o Google Cloud. Estas ofrecen un mayor número de servicios y permiten desplegar todas las partes de una aplicación. No obstante, su configuración es más avanzada y requiere un mayor conocimiento de infraestructuras.

Fuente Docker: <https://docs.docker.com/get-started/docker-overview/>

En el despliegue de una aplicación, también es importante gestionar correctamente las claves y credenciales del sistema. Datos como la clave de la API o la contraseña de la base de datos no deben escribirse directamente en el código. Durante el desarrollo local, esta información suele guardarse en un archivo `.env`, que contiene variables de entorno utilizadas por la aplicación. En producción, estas variables se configuran directamente desde la plataforma de despliegue, como Vercel o Render. De esta forma, se evita exponer información sensible.

Nota: En los 2 últimos puntos, otra vez lo mismo; podría poner fuentes de vite, docker, docker compose, vercel, render, railway, aws... No sé si poner una fuente web de absolutamente todo lo que mencione o solo de lo más importante. Al final, en el marco teórico también habrá multitud de fuentes por todos lados. De momento las voy poniendo como links y al final ya se colocan como siempre.

2.4. Limitaciones de las soluciones existentes

Las herramientas analizadas anteriormente presentan funcionalidades muy útiles para los jugadores de LoL, pero suelen estar enfocadas en una necesidad concreta. Algunas aplicaciones se centran principalmente en la consulta de perfiles e historiales de partidas, mientras que otras están más enfocadas en builds, estadísticas de campeones o seguimiento de jugadores profesionales. Esto provoca que el usuario tenga que consultar varias herramientas diferentes para obtener la información que necesita.

Otra limitación muy común es la sobrecarga de información. Muchas de estas plataformas muestran una gran cantidad de datos, estadísticas, recomendaciones, comparativas y elementos visuales en una misma interfaz. Aunque esto puede ser útil para jugadores muy avanzados, también puede

dificultar la búsqueda de información concreta para usuarios menos experimentados.

Recientemente, las herramientas de historiales o análisis de partidas han añadido funciones con inteligencia artificial que asignan una puntuación al rendimiento del jugador en cada partida. Sin embargo, esta puntuación no suele basarse en criterios claros y, muchas veces, no concuerda con lo acontecido durante la partida. Al ser un entorno con mucho tráfico y una fuerte competencia entre plataformas, estas herramientas tienden a incorporar funcionalidades llamativas con el fin de diferenciarse y atraer más usuarios. No obstante, estas funciones no llegan a satisfacer una necesidad real del usuario, sino que funcionan más como añadidos para atraer público que como herramientas realmente útiles.

En algunos casos, el problema no es solo la cantidad de información, sino también la forma en la que se presenta. Al consultar una build, algunas plataformas muestran varias combinaciones posibles de objetos en lugar de ofrecer una recomendación más directa. Aunque esta variedad puede ser útil para jugadores experimentados, no siempre resulta adecuada para los usuarios que buscan una respuesta rápida y sencilla.

Estas limitaciones muestran que, aunque existen herramientas muy completas en el ecosistema de LoL, no siempre están preparadas para todo tipo de usuarios. Por ello, todavía existe espacio para nuevas aplicaciones más sencillas, directas y centradas en nuevos casos de uso.

2.5. Aportación de este trabajo

La aportación principal de este trabajo consiste en el desarrollo de una aplicación web full-stack orientada al análisis y consulta de datos de LoL. Para ello, se integran en una misma herramienta varias funcionalidades habituales en este tipo de aplicaciones, como la búsqueda de jugadores, la consulta de información básica del perfil, el historial de partidas, los detalles de cada partida, el estado de la partida actual y el seguimiento de builds de jugadores profesionales.

Este trabajo no busca sustituir a plataformas consolidadas como OP.GG o Blitz.gg, sino ofrecer una alternativa más sencilla y centralizada. La idea principal es reunir en una misma aplicación la información básica que un jugador puede necesitar, evitando que tenga que acudir a varias herramientas para satisfacer sus necesidades.

Además, el trabajo aporta una solución técnica completa mediante el diseño e implementación de una arquitectura formada por frontend, backend y base de datos. El frontend se encarga de la interacción con el usuario,

el backend centraliza la lógica de negocio y la comunicación con la API de Riot, y la base de datos permite almacenar la información consultada. Esta organización facilita el mantenimiento del sistema y permite reducir llamadas repetidas a la API reutilizando los datos ya almacenados.

También se puede considerar una aportación metodológica, ya que el proyecto se desarrolla de forma iterativa e incremental. En lugar de intentar construir desde el inicio una aplicación completa con todas las funcionalidades posibles, se parte de un MVP sobre el que se van añadiendo nuevas características. Este enfoque permite validar progresivamente cada incremento y dejar preparada una base sobre la que incorporar futuras funcionalidades.

Nota: Esto lo he puesto porque me lo dijiste en el correo del índice, pero me suena a paja. ¿Cómo lo ves?

Por todo esto, este trabajo aporta una aplicación centralizada con un enfoque más sencillo, accesible y adaptado a los usuarios menos experimentados. No se basa únicamente en la cantidad de datos mostrados, sino en la forma de organizarlos y presentarlos para facilitar la experiencia del usuario.

Capítulo 3

Marco teórico

En este capítulo se presentan los conceptos necesarios para comprender el desarrollo técnico del proyecto. No se pretende realizar un estudio exhaustivo del videojuego, sino introducir el vocabulario principal relacionado con LoL y con la obtención de datos desde servicios externos. El objetivo es proporcionar una base conceptual que facilite la lectura de los siguientes capítulos.

3.1. Conceptos básicos de League of Legends

LoL es un videojuego competitivo del género MOBA en el que dos equipos formados por cinco jugadores compiten entre sí. Cada jugador selecciona un campeón y ocupa una posición determinada dentro de un mapa. Durante la partida, los jugadores deben tomar decisiones relacionadas con su posición en el mapa, la coordinación del equipo, la compra de objetos o el orden de mejora de sus habilidades.

Gran parte de los conocimientos necesarios para tomar buenas decisiones a lo largo de la partida se obtienen con la experiencia. No obstante, algunas decisiones, como qué campeón escoger, qué objetos comprar o el orden de mejora de sus habilidades, pueden saberse rápidamente analizando los datos disponibles en la API de Riot.

Para satisfacer la necesidad del jugador de tener una fuente fiable de información a la que acudir, se comenzaron a desarrollar aplicaciones de análisis de datos de videojuegos. Estas aplicaciones utilizan información procedente de partidas, jugadores, campeones y otros elementos del juego.

Por todo esto, para entender el funcionamiento de este trabajo, es necesario introducir algunos conceptos del videojuego:

- **Campeón:** Personaje jugable seleccionado por cada jugador al inicio de una partida. Cada campeón tiene habilidades y características

diferentes. Actualmente, el videojuego cuenta con 172 campeones diferentes.

Nota: Para saber el número de campeones es meterte al videojuego y mirarlo, como pongo eso de fuente? porque el videojuego no es ninguna web ni nada para citar

- **Rol o posición:** Función de un jugador dentro de un equipo. En LoL hay cinco posiciones diferentes: Top, Jungla, Mid, Bot y Support. La API de Riot trata las posiciones como TOP, JUNGLE, MIDDLE, BOTTOM y UTILITY respectivamente. Cada una define las responsabilidades de un jugador dentro del equipo y la zona del mapa de la que se encarga.
- **Build:** Conjunto de objetos comprados por un jugador durante la partida. Una build está formada por un máximo de 6 objetos. Cada objeto tiene un coste y otorga estadísticas diferentes al campeón que lo posee. Los objetos elegidos y el orden en el que se compren son una de las decisiones más importantes a la hora de ganar una partida. Hoy en día, el LoL cuenta con más de 200 objetos diferentes.
- **Parche:** Versión del juego publicada por Riot Games. Cada parche puede introducir cambios en campeones, objetos o sistemas dentro del videojuego. Suele haber un parche cada dos semanas con el fin de equilibrar los diferentes campeones para que ninguno sobresalga por encima del resto.
- **META:** Conjunto de campeones, objetos o estrategias más eficientes en el momento actual del juego. El META cambia constantemente debido a los numerosos parches que Riot realiza al LoL, por lo que, para tener un buen rendimiento, es esencial conocer el META en todo momento.

Nota: No sé si choca con la primera motivación donde ya explico la existencia del meta.

- **Cola:** Aunque la cola o modo de juego principal de LoL siempre ha sido la Grieta del Invocador, el videojuego dispone de modos de juego extra donde cambian algunas características, como el número de jugadores, el mapa o la selección del campeón. El competitivo está basado en el modo principal, no obstante, la API de Riot muestra datos de todo tipo de partidas. Aunque las recomendaciones están basadas únicamente en las partidas del modo principal, en el historial de un jugador se verán reflejadas partidas de cualquier modo.

- **Winrate y pickrate:** Las estadísticas más importantes de los videojuegos competitivos han sido siempre el *winrate* y el *pickrate*. La tasa de elección de un campeón o compra de un objeto, junto con su tasa de victorias, permite conocer el estado actual del mismo. Los mejores campeones u objetos del META suelen ser los que poseen la mayor tasa de victorias y de elección.
- **Jugador profesional:** Una buena referencia a la hora de conocer los mejores campeones u objetos del momento es fijarse en los jugadores profesionales. Estos, dedican su jornada laboral a estudiar el videojuego y son una fuente muy fiable de información.

Teniendo estos conceptos en cuenta, una aplicación de análisis de datos de LoL debe proporcionar la información necesaria para conocer el META. Por ello, este tipo de aplicaciones muestran historiales con detalles sobre las partidas jugadas, estadísticas por campeón para conocer que campeones tienen mayor tasa de victorias o son más jugados en el momento e incluso builds recomendadas para estos campeones.

Nota: Puedo entrar en más detalle, pero creo que con esto es suficiente. Tú me dices, tu opinión es muy importante aquí porque es de alguien que jamás ha jugado al juego, igual que el tribunal

3.2. Obtención y tratamiento de datos

3.2.1. API de Riot

Una *Application Programming Interface* (API) es un conjunto de reglas o protocolos que permite que diferentes aplicaciones se comuniquen entre sí. En este contexto, una API externa es un servicio proporcionado por un tercero que permite consultar o enviar información desde otra aplicación.

Fuente API: <https://www.ibm.com/es-es/think/topics/api> Fuente 2 API: <https://aws.amazon.com/es/what-is/api/>

En este proyecto, la API externa utilizada es la API de Riot, ya que proporciona todos los datos necesarios para obtener información de jugadores, partidas y otros elementos del videojuego. Esta API sigue un modelo REST, uno de los más populares actualmente. Las API REST se comunican haciendo uso del protocolo HTTP.

En la aplicación desarrollada en este trabajo, el frontend no se comunica directamente con la API de Riot. Las peticiones se realizan al backend, que actúa como intermediario entre el usuario y la API de Riot.

Cuando se realiza una petición a la API de Riot, esta devuelve una respuesta con información estructurada en formato *JavaScript Object Notation* (JSON). Este formato permite representar los datos siguiendo una estructura de pares clave-valor.

Riot representa estas respuestas mediante *Data Transfer Objects* (DTOs), objetos utilizados para transportar datos. Como se muestra en la Figura 3.1, cada llamada a la API devuelve un objeto formado por diferentes campos, dependiendo de la consulta realizada.

ACCOUNT-V1

GET

/riot/account/v1/accounts/by-puuid/{puuid}

Jump to Inputs

RESPONSE CLASSES

Return value: AccountDto

AccountDto

NAME	DATA TYPE	DESCRIPTION
puuid	string	Encrypted PUUID. Exact length of 78 characters.
gameName	string	This field may be excluded from the response if the account doesn't have a gameName.
tagLine	string	This field may be excluded from the response if the account doesn't have a tagLine.

Figura 3.1: API de Riot. DTO

Fuente API Riot: <https://developer.riotgames.com/>

3.2.2. Riot ID

En LoL, los jugadores se identifican mediante el Riot ID. Este identificador está formado por dos partes: el nombre de jugador y una etiqueta. Un ejemplo de Riot ID sería `iFran28#EUW`, donde `iFran28` es el nombre del jugador y `EUW` la etiqueta. Esto permite que puedan existir nombres repetidos, siempre que usen una etiqueta diferente.

Conocer esta estructura de nombrado es importante ya que la API de Riot utiliza el nombre y la etiqueta para localizar una cuenta. A partir de estos datos, la API devuelve su propio identificador interno, el PUUID, que se utiliza posteriormente para realizar consultas relacionadas con el jugador en cuestión.

A la hora de buscar un jugador, además del Riot ID, se debe incluir la plataforma o servidor en la que el jugador está registrado.

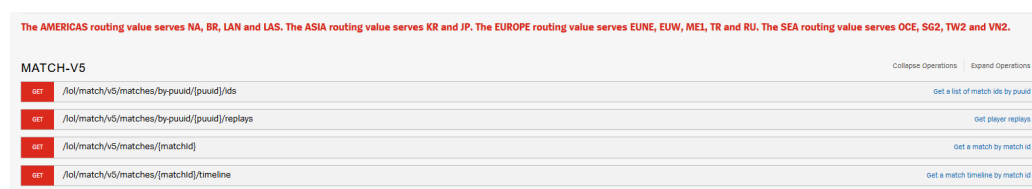
Fuente Riot ID: <https://www.riotgames.com/en/news/summoner-name-riot-ID>

3.2.3. Plataformas y regiones

La API de Riot no utiliza una única dirección para todas las peticiones. Para realizar una llamada a la API correctamente, es necesario seleccionar el host adecuado en función del endpoint que se quiere consultar. Riot utiliza valores de enrutamiento que pueden corresponder a una plataforma o a una región.

Las plataformas representan los servidores del videojuego, como EUW1, para el oeste de Europa, o KR, para Corea del Sur. Por otro lado, las regiones agrupan varias plataformas, como EUROPE, AMERICAS, ASIA o SEA.

Cada endpoint de la API de Riot muestra la plataforma o región sobre la que se debe hacer la petición. Como se puede ver en la 3.2, en el caso del endpoint MATCH-V5, la documentación oficial muestra que para las llamadas realizadas a dicho endpoint, se deben utilizar las regiones.



The AMERICAS routing value serves NA, BR, LAN and LAS. The ASIA routing value serves KR and JP. The EUROPE routing value serves EUNE, EUW, ME1, TR and RU. The SEA routing value serves OCE, SG2, TW2 and VN2.	
MATCH-V5	
GET	/lol/match/v5/matches/by-puuid/{puuid}/ids
GET	/lol/match/v5/matches/by-puuid/{puuid}/replays
GET	/lol/match/v5/matches/{matchId}
GET	/lol/match/v5/matches/{matchId}/timeline

Figura 3.2: Endpoint MATCH-V5

Nota: Esto es raro, dentro de cada endpoint (hay muchísimos), te dice si debes usar europe/asia/america/sea o el servidor concreto EUW1/KR/NA1... En algunas te dice que puedes usar cualquiera y funciona, y en otras te dice que debes usar una region o servidor concreto.

Al final la region es una forma de agrupar servidores. He puesto el ejemplo de match-v5 porque al final son partidas y es el más básico pero se ve enana la aclaración oficial arriba (No sé si quitarla). También hay tablas con todos los servidores y regiones, pero veo excesivo poner eso, aunque no me cuesta nada.

He usado también la de match porque arriba te dice a que region corresponde cada plataforma/servidor concreto.

Por tanto, una petición al endpoint MATCH-V5 para consultar una partida de un jugador perteneciente al servidor europeo tendría como base la siguiente URL:

`https://europe.api.riotgames.com/lol/match/v5/matches/{matchId}`

Esto implica que la aplicación debe distinguir entre la plataforma seleccionada por el usuario, por ejemplo EUW1, y la región asociada que se debe utilizar en determinados endpoints, en este caso EUROPE. No obstante, como cada plataforma pertenece a una región concreta, la aplicación puede obtener esta región internamente. Por ello, el usuario solo debe indicar la plataforma a la que pertenece el jugador que desea buscar.

Nota: Cada que hablo de EUW, MATCH-V5... debería dejarlo así o ponerlo como la url entre `/texttt`

Fuente plataformas y regiones: <http://developer.riotgames.com/docs/lol#routing-values>

Fuente api de riot: <https://developer.riotgames.com/apis>

3.2.4. Partidas y participaciones

La API de Riot devuelve la información de una partida en un objeto llamado `MatchDto`. Este objeto contiene, por un lado, metadatos de la partida, como su identificador, y por otro, información general sobre la misma, como la duración, el modo de juego o la versión del parche.

Además de estos datos generales, una partida tiene información de todos los jugadores que han participado en ella. Riot representa esta información mediante participaciones. En una partida estándar hay diez participaciones, una por cada jugador.

Cada participación guarda los datos de un jugador dentro de esa partida. Entre ellos se encuentran datos relevantes como el campeón utilizado, el rol, las estadísticas principales y los objetos comprados. Por este motivo, las participaciones son uno de los elementos más importantes para este trabajo.

Capítulo 4

Análisis del problema

En este capítulo se analiza el problema que pretende resolver la aplicación. Para ello, se define el alcance de la solución, se establecen los requisitos del sistema y se describen los componentes que tendrá la aplicación. Además, se presenta la arquitectura general del sistema junto con los flujos de información entre el usuario, los componentes y los servicios externos.

Nota: en el guión que me diste ponía **“Análisis del problema y metodología empleada”**, no sé como poner lo de metodología porque en el apartado 6 va de eso y en la introducción ya lo nombré, se me quedaba muy repetitivo.

4.1. Planteamiento del problema

El problema que aborda este trabajo surge de la complejidad de LoL y de la necesidad que tienen los jugadores de consultar información actualizada de forma rápida y sencilla. LoL es un videojuego competitivo con una gran cantidad de variables: campeones, roles, objetos, parches y estrategias de juego diferentes. Esta complejidad hace que los jugadores necesiten mantenerse actualizados para tomar las mejores decisiones dentro de una partida.

Además de la necesidad de información constante, la dificultad del videojuego genera una barrera de entrada elevada para los nuevos jugadores. Riot Games ha reconocido en varias publicaciones oficiales que el videojuego presenta una curva de aprendizaje muy alta para los nuevos jugadores. Con el fin de reducirla, Riot ha tomado medidas como facilitar el videojuego y crear controles más amigables para nuevos usuarios.

Fuente barrera de entrada 1: <https://www.leagueoflegends.com/en-us/news/dev/lol-pls-preseason-2023-jungle-changes/> Fuente barrera de

entrada 2: <https://www.leagueoflegends.com/en-us/news/dev/dev-wasd-controls-are-on>

Actualmente, existen distintas herramientas que permiten obtener datos sobre perfiles, partidas, campeones o builds. Sin embargo, muchas de ellas están muy especializadas o muestran una cantidad excesiva de información en pantalla. Esto puede ser útil para jugadores experimentados, pero dificulta la experiencia de los nuevos usuarios, que buscan una respuesta rápida y sencilla.

Por todo esto, el problema no es la ausencia de datos, sino la dificultad de centralizarlos, organizarlos y presentarlos de forma clara para que el usuario novato pueda consultarlos fácilmente.

Desde el punto de vista técnico, el problema consiste en obtener la información desde una fuente externa, procesarla, almacenarla y mostrarla de forma comprensible al usuario. La API de Riot proporciona los datos necesarios, pero se encuentran repartidos en distintos endpoints y se devuelven en estructuras que no están pensadas para el usuario final. Esto hace necesario plantear una solución que actúe como intermediaria entre los datos disponibles en la API de Riot y el usuario que desea consultarla.

La solución propuesta para este problema es el desarrollo de una aplicación web que consulte, procese y organice los datos obtenidos de la API de Riot para que el usuario pueda acceder a la información de forma centralizada, clara y sencilla.

4.2. Requisitos del sistema

A partir del problema planteado, se definen los requisitos que debe cumplir la aplicación. Estos requisitos se dividen en requisitos funcionales, que describen qué debe hacer el sistema, requisitos no funcionales, que definen cómo debe operar el sistema, y unos requisitos funcionales de ampliación, con mejoras funcionales para ampliar el sistema en incrementos posteriores.

A continuación, se muestran los requisitos definidos para el sistema.

ID	Requisito	Descripción
RF1	Búsqueda de perfil	El sistema debe permitir buscar un jugador mediante su Riot ID y la plataforma correspondiente.
RF2	Consulta de perfil	El sistema debe mostrar la información básica del jugador buscado.
RF3	Historiales de partidas	El sistema debe permitir consultar el historial de partidas recientes de un jugador.
RF4	Detalle de partida	El sistema debe permitir consultar los detalles de una partida concreta del historial.
RF5	Estado de la partida actual	El sistema debe mostrar si el jugador se encuentra actualmente en partida y mostrar información básica de la misma.
RF6	Campeones	El sistema debe permitir ver todos los campeones disponibles.
RF7	Builds de profesionales	El sistema debe permitir consultar builds recientes utilizadas por jugadores profesionales para un campeón concreto.
RF8	Información de objetos	El sistema debe mostrar la información básica de los objetos al interactuar con ellos.
RF9	Información actualizada	El sistema debe permitir actualizar la información de jugadores y partidas consultando la API de Riot.

Tabla 4.1: Requisitos funcionales.

Nota: Es el formato de tabla de manuel, no sé si está bien, pero como me pasó la plantilla, lo reutilicé (cambiándole que la última columna tenga multilínea). Pd: Luis nos hizo en su día hacer también requisitos del sistema. Ej: el sistema operativo será windows. Yo no lo he puesto porque lo considero paja, pero podría ponerlos.

ID	Requisito	Descripción
RNF1	Usabilidad y accesibilidad	La interfaz debe ser clara y sencilla, permitiendo que usuarios con distinto nivel de conocimiento puedan consultar la información sin dificultad.
RNF2	Persistencia de datos	El sistema debe almacenar la información para reutilizar datos ya consultados y reducir el número de llamadas a la API externa.
RNF3	Gestión de errores	El sistema debe gestionar los errores derivados de la API externa.
RNF4	Separación de responsabilidades	El sistema debe separar la interfaz de usuario, la lógica de negocio y la persistencia de datos.
RNF5	Despliegue local	El sistema debe poder ejecutarse en un entorno local separando sus componentes principales.

Tabla 4.2: Requisitos no funcionales.

ID	Requisito	Descripción
RFA1	Estadísticas de campeones	El sistema debe mostrar estadísticas relacionadas con campeones.
RFA2	Estadísticas de jugadores	El sistema debe mostrar estadísticas relacionadas con el rendimiento de los jugadores.
RFA3	Actualización automática	El sistema debe automatizar la actualización de información mediante tareas programadas periódicamente.
RFA4	Recomendación de builds	El sistema debe mostrar una build recomendada para cada campeón basándose en las partidas jugadas con dicho campeón.

Tabla 4.3: Requisitos funcionales de ampliación.

Nota: en su día, hice los objetivos de ampliación, que se traducen aquí como requisitos funcionales de ampliación. El tema es que

puse como objetivo de ampliación ”Preparar el paso a producción”, básicamente que la web sea accesible en internet con un dominio, un hosting... La cosa es; eso es un requisito no funcional. ¿Lo meterías en no funcionales aunque sea de ampliación, lo ignorarías, o harías una tabla de requisitos no funcionales de ampliación?

4.3. Arquitectura general y componentes del sistema

La aplicación se basa en una arquitectura cliente-servidor formada por varios componentes principales. En la Figura 4.1 se muestra el diagrama de componentes de alto nivel del sistema.

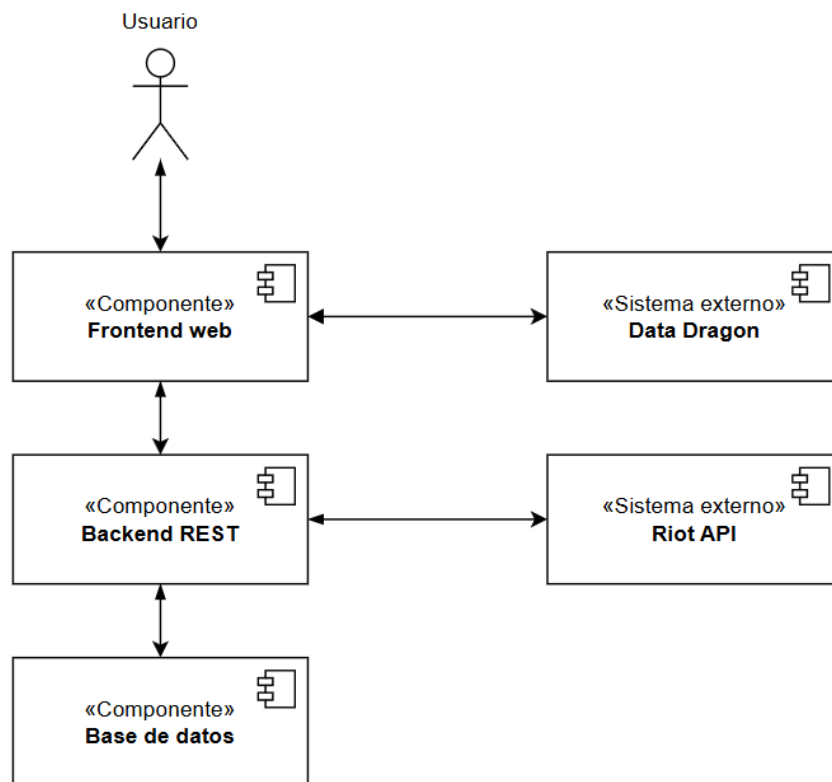


Figura 4.1: Diagrama de componentes de alto nivel

El usuario interactúa desde un navegador web, donde se ejecuta el frontend. El frontend, a su vez, no interactúa directamente con la API de Riot,

sino que se comunica con el backend mediante peticiones HTTP y carga los datos estáticos proporcionados por Data Dragon.

El backend es el componente central del sistema. Su función es recibir las peticiones del frontend, aplicar la lógica de negocio necesaria, consultar la base de datos en busca de información ya almacenada y comunicarse con la API de Riot para obtener o actualizar datos cuando sea necesario.

La base de datos se utiliza para almacenar la información que el sistema puede reutilizar. Se guardan los datos relacionados con jugadores, partidas, participaciones, campeones, builds y jugadores profesionales.

Esta separación en componentes permite que cada parte del sistema tenga una responsabilidad concreta. El frontend se encarga de la interacción con el usuario, el backend de la lógica de negocio y el acceso a los datos, la base de datos de almacenar la información y los servicios externos de proporcionar los datos y recursos necesarios para el funcionamiento de la aplicación.

4.4. Flujo de información

Una vez se ha definido la arquitectura general, se van a describir los principales flujos de información del sistema. En este apartado no se pretende explicar la implementación o diseño de la aplicación, sino representar de forma general cómo se obtiene, organiza y muestra la información.

El primer flujo corresponde a la consulta de información de un jugador, ya que es una de las funciones principales de la aplicación. El usuario introduce el Riot ID y la plataforma correspondiente desde el frontend. A partir de estos datos, la solicitud se envía al backend, que se encarga de procesarla y comprobar si existe información almacenada en la base de datos previamente.

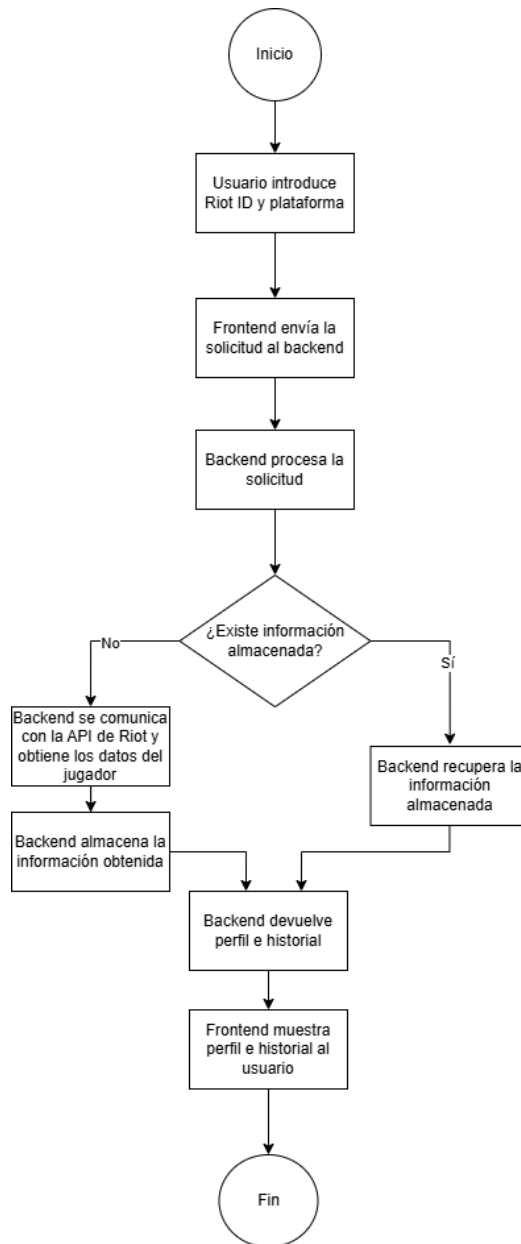


Figura 4.2: Diagrama de flujo. Perfil de usuario

Como se muestra en la Figura 4.2, si la información ya se encuentra almacenada, el backend la recupera y devuelve al frontend. En caso contrario, consulta a la API de Riot para obtener los datos del jugador, almacena la información obtenida y devuelve al frontend el perfil junto con el historial de partidas, que se mostrará al usuario.

El segundo flujo representa la consulta de builds recientes de jugadores

profesionales. En este caso el usuario selecciona un campeón desde el frontend y se vuelve a enviar la solicitud al backend. A diferencia del flujo anterior, la consulta de builds se realiza sobre las partidas ya almacenadas en la base de datos previamente.

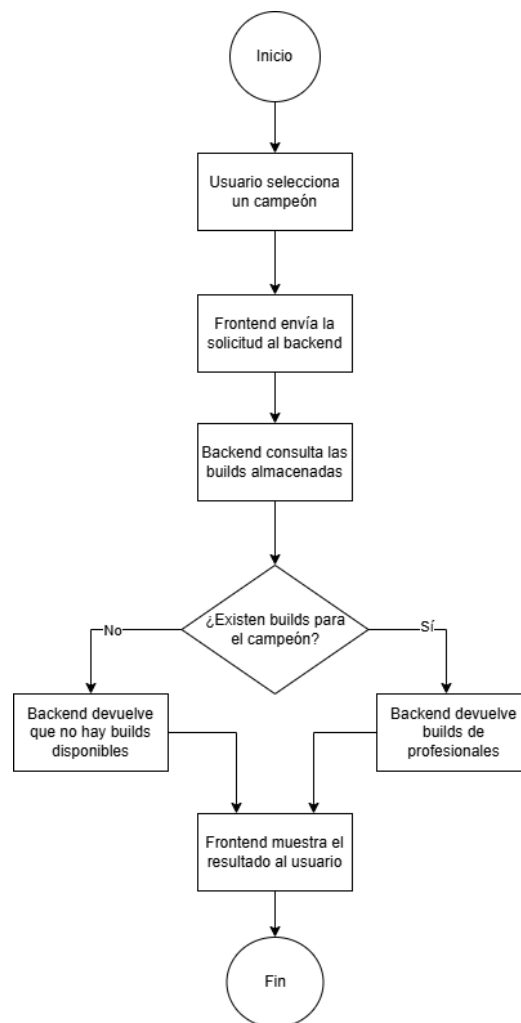


Figura 4.3: Diagrama de flujo. Builds de profesionales

Como se muestra en la Figura 4.3, el backend comprueba si existen builds disponibles para el campeón seleccionado. Si existen, devuelve las builds de los profesionales al frontend para que se muestren al usuario. Si no hay builds disponibles, el backend devuelve una respuesta vacía y el frontend muestra dicho resultado.

Estos dos flujos representan las dos funciones principales de la aplicación: consultar información de un jugador concreto y consultar información

relacionada con campeones y jugadores profesionales. En el primer caso, la información se obtiene directamente desde la API de Riot y puede almacenarse para futuras consultas. En el segundo caso, las builds profesionales se obtienen a partir de información ya almacenada y procesada por el sistema.

Capítulo 5

Diseño del sistema

- 5.1. Diseño de la arquitectura software
- 5.2. Diseño del backend
- 5.3. Diseño del frontend
- 5.4. Diseño de la base de datos
- 5.5. Diseño de la API REST
- 5.6. Diseño del despliegue

Capítulo 6

Desarrollo incremental

- 6.1. Resumen general de las iteraciones
- 6.2. Iteración 1
- 6.3. Iteración 2

Capítulo 7

Evaluación de la aplicación final

- 7.1. Entorno de despliegue
- 7.2. Casos de uso y validación funcional
- 7.3. Comprobación del cumplimiento de requisitos
- 7.4. Análisis crítico del desarrollo realizado
- 7.5. Limitaciones del sistema

Capítulo 8

Conclusiones y trabajo futuro

- 8.1. Conclusiones
- 8.2. Grado de cumplimiento de los objetivos
- 8.3. Aportaciones del trabajo
- 8.4. Limitaciones actuales
- 8.5. Líneas de trabajo futuro

Bibliografía

- [1] D. G. Pai *et al.*, “Explainable ai in agriculture: review of applications, methodologies, and future directions,” *Engineering Research Express*, vol. 7, no. 3, p. 032202, 2025.
- [2] Ö. Turgut, İ. Kök, and S. Özdemir, “Agroxai: Explainable ai-driven crop recommendation system for agriculture 4.0,” *arXiv preprint arXiv:2412.16196*, 2024.
- [3] A. Yenikar, V. P. Mishra, M. Bali, and T. Ara, “An explainable ai-based hybrid machine learning model for interpretability and enhanced crop yield prediction,” *MethodsX*, vol. 15, p. 103442, 2025.
- [4] I. Malashin, V. Tynchenko, A. Gantimurov, V. Nelyub, A. Borodulin, and Y. Tynchenko, “Predicting sustainable crop yields: Deep learning and explainable ai tools,” *Sustainability*, vol. 16, no. 21, p. 9437, 2024.
- [5] S. Murindanyi, J. Nakatumba-Nabende, R. Sanya, R. Nakibuule, and A. Katumba, “Enhanced infield agriculture with interpretable machine learning approaches for crop classification,” *arXiv preprint arXiv:2408.12426*, 2024.
- [6] P. Florek and A. Zagdański, “Benchmarking state-of-the-art gradient boosting algorithms for classification,” *arXiv preprint arXiv:2305.17094*, 2023.
- [7] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, pp. 3146–3154, 2017.

- [9] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, pp. 4765–4774, 2017.
- [10] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 618–626, 2017.
- [11] The Apache Software Foundation, “Apache parquet.” <https://parquet.apache.org/>, 2024. Accessed: 2024-06-01.
- [12] B. Courty, V. Schmidt, *et al.*, “Codecarbon: Track and track carbon emissions from machine learning computing,” 2024.
- [13] S. Mallat, *A wavelet tour of signal processing: the sparse way*. Academic press, 3rd ed., 2008.
- [14] G. R. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt, and A. O’Leary, “Pywavelets: A python package for wavelet analysis,” *Journal of Open Source Software*, vol. 4, no. 36, p. 1237, 2019.